

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Safe Guard	Documentation quality	High	
Timeline	2025-03-24 through 2025-03-28	Test quality	High	
Language	Solidity	Total Findings	7	Fixed: 5 Acknowledged: 1 Mitigated: 1
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Specification	Hypernative Safe Multisig Guard	Medium severity findings ⓘ	2	Fixed: 1 Mitigated: 1
Source Code	Hypernative-Labs/hypernative-guard #5ae5958 Hypernative-Labs/hypernative-guard #7788a72	Low severity findings ⓘ	3	Fixed: 2 Acknowledged: 1
Auditors	- Jennifer Wu Auditing Engineer - Leonardo Passos Senior Research Engineer - Michael Boyle Auditing Engineer	Undetermined severity findings ⓘ	1	Fixed: 1
		Informational findings ⓘ	1	Fixed: 1

Summary of Findings

The `HypernativeGuard` smart contract is a Safe transaction guard designed to enforce approval policies for wallet transactions. It integrates with the Safe wallet by verifying transactions against pre-approved hashes (regular, nonce-free, and function call hashes) and leverages a timelock mechanism to add a delay on sensitive operations such as revocations. The contract extends both a base transaction guard and OpenZeppelin's `AccessControl`, ensuring that only authorized entities can approve or revoke transactions. Additionally, it supports modular extensions via a policy extension interface, allowing for customizable and layered transaction validation.

Quantstamp reviewed the Hypernative Safe Guard implementation and identified 7 findings and 2 suggestions. Among these findings is a vulnerability in the timelock mechanism, which can be bypassed, even when active, by changing the timelock delay, allowing instant guard changes (**HNG-1**). Additionally, the handling of policy extensions could introduce transaction reverts if they are malicious or faulty (**HNG-2**). We recommend the client carefully consider all identified issues.

Fix Review: The client resolved all issues by either implementing the fixes or acknowledging them. Notably, the client removed the timelock duration change functionality in **HNG-1** by fixing the duration to one day, and changed policy extension removals to the keeper in **HNG-2**.

ID	DESCRIPTION	SEVERITY	STATUS
HNG-1	Timelock Duration Can Be Changed While Timelock Is Triggered	● Medium ⓘ	Fixed
HNG-2	Safe Denial of Service Through Malicious or Inefficient Policy Extensions	● Medium ⓘ	Mitigated
HNG-3	Bypass Safe Guardian Checks Using Module Execution	● Low ⓘ	Acknowledged
HNG-4	Hypernative Guard Is Not Suitable for Reuse	● Low ⓘ	Fixed
HNG-5	Missing Ability to Revoke Keeper Role	● Low ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
HNG-6	No Check for Duplicate Policies	• Informational ⓘ	Fixed
HNG-7	Policies Are Not Provided with the Transaction Signatures	• Undetermined ⓘ	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

i **Disclaimer**

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `Hypernative-Labs/hypernative-guard/5ae5958d603671135b753084320df432f7b50cd3`

Included Paths: `src/`

Repo: `https://github.com/Hypernative-Labs/hypernative-guard`

Repo: `https://github.com/Hypernative-Labs/hypernative-guard`

Operational Considerations

- The Safe owners should ensure that all policy extensions undergo security review before installation. Malicious or inefficient extensions can revert or excessively consume gas, resulting in permanent denial-of-service of the Safe. Policy extension contracts must implement the `IGuardPolicyExtension` interface securely.
- Modules installed on Safe versions 1.3 and 1.4 bypass guardian checks. The Safe owners should exercise caution when enabling new modules, as they can execute arbitrary transactions without guardian checks.
- The Safe owners should confirm that the `timelockBlock` is consistently set to the intended duration (currently initialized to 1 day). If accidentally overwritten or misconfigured (e.g., reset to zero), the timelock protection can be bypassed, allowing immediate guard changes without the intended delay.
- The Safe owners should carefully assign keeper roles only to trusted entities, as Safe currently lacks the functionality to revoke keeper permissions. Until revocation capability is added, limit keeper privileges strictly to secure, reliable addresses.
- The keeper should exercise caution when approving function selectors (`getFunctionCallHash()`), as the current hashing mechanism only considers the first 4 bytes (the function selector) and ignores transaction parameters. Approving commonly used selectors like `transfer()` or `transferFrom()` for a token contract (e.g., USDC) could unintentionally authorize transactions with arbitrary parameters.
- The keeper should ensure that transaction hashes are correctly calculated and submitted to the contract.

Fix Review: Following the fix review,

- Keepers can remove faulty or malicious policy extensions. Note that permanent denial-of-service can still occur from policy extensions if access to keeper roles is lost.
- The timelock mechanism is fixed to 1 day. It is not possible to update this value.
- Safe can revoke the keeper role from existing addresses.
- When revoking a Hypernative Safe Guard, the Safe owners should call `disableTimelock()` separately to disable the activated timelock following a successful revoking action.

Key Actors And Their Capabilities

- Keepers (`KEEPER_ROLE`): Initially assigned to the keeper address defined during deployment. Addresses with this role can approve and revoke Safe transaction hashes. This role can also remove existing policy extensions.
- Safe: Manages sensitive operations such as adding policy extensions, triggering timelock sequences, and approving or revoking other addresses as keepers.
- Policy Extension Contracts: Implement the defined policy interface and provide additional custom validation logic for transactions, allowing the guard to enforce extended or evolving security policies beyond its built-in checks.

Findings

HNG-1

Timelock Duration Can Be Changed While Timelock Is Triggered

• Medium ⓘ Fixed

✓ Update

The client provided the fix in `4ed6793839a2b17d8a79732a7b35284bee8cdea8` by removing `changeTimelockDuration()` and hardcoding the delay to 1 day from when it is activated.

File(s) affected: `HypernativeGuard.sol`

Description: The `HypernativeGuard` contract uses a timelock mechanism to prevent an attacker from immediately removing the guard from the Safe contract. However, a compromised multisig wallet can get around the timelock and revoke the guard by calling `changeTimelockDuration()` to set `timelockDuration` to zero, followed by a call to `activateTimelock()`. This will set `timelockBlock` to the current block timestamp -- allowing the attacker to revoke the guard in the next block. This attack assumes that a Keeper has approved the Safe to call `changeTimelockDuration()`, which is not approved by default like `activateTimelock()` and `disableTimelock()`.

Recommendation: Disable the ability to call `changeTimelockDuration()` while the timelock is triggered and consider enforcing a reasonable amount of time for the timelock period.

HNG-2

Safe Denial of Service Through Malicious or Inefficient Policy Extensions

• Medium ⓘ Mitigated

✓ Update

The client mitigated in `08a4c4ab4155e490d401f38c65f7d30d939366fe` by changing the `removePolicy()` functionality to require keeper access rather than Safe access. However, if the existing policy is inefficient or faulty and access to the keeper is lost (or malicious), the keeper can block the Safe from invoking transactions.

File(s) affected: `HypernativeGuard.sol`

Description: When the Safe guard calls each policy extension's `checkPolicy()`, a malicious or erroneous extension can revert or consume excessive gas, causing the entire Safe transaction to fail. Because adding or removing policy extensions also requires a successful Safe transaction, a malicious extension can permanently lock the Safe by denying all transactions, including the one required to remove it.

Recommendation: Consider implementing a recovery mechanism (e.g., via a keeper or a privileged process) that can remove or disable malicious policy extensions when normal Safe transactions are blocked, or be very careful when adding extensions to ensure they are trusted and non-malicious. Since a Safe Guard has full power to block Safe transaction execution, a **broken Guard** can cause a denial of service for a Safe. Also, consider setting a reasonable maximum amount of policy extensions that can be used.

HNG-3 Bypass Safe Guardian Checks Using Module Execution • Low ⓘ Acknowledged

Update

The client acknowledged and is aware of Safe Guardian Check limitations.

File(s) affected: `HypernativeGuard.sol`

Description: Safe can add modules that introduce functionalities beyond the core Safe capabilities. While these modules enhance the utility of Safe by allowing custom features, they can also execute arbitrary transactions once installed. Currently, in Safe versions 1.3 and 1.4, transactions initiated by these modules do not undergo guardian checks, presenting a loophole if the `enableModule()` function is permitted by the guardian settings. This lack of guardian oversight for module-executed transactions can be used to bypass guardian checks.

Exploit Scenario:

1. The guardian configuration permits the use of `enableModule()`.
2. Safe owners collectively decide to add a new module by invoking `enableModule()`.
3. The installed module performs arbitrary transactions without undergoing the HyperNative Guardian's checks.

Recommendation: This limitation exists for Safe versions 1.4 and earlier. Modules in Safe versions 1.3 and 1.4 do not invoke guardian checks. Safe version 1.5 will introduce support for **guardian checks** on module-executed transactions. Exercise caution when enabling new modules for Safe accounts running version 1.4 or earlier.

HNG-4 Hypernative Guard Is Not Suitable for Reuse • Low ⓘ Fixed

Update

The client addressed in `aa9f1fc77b7dca2aa5c659289260177749ccf8bd` by resetting the timelock state variables when invoking `disableTimelock()`. Note that these state variables are not automatically reset after successfully revoking the Safe Guard. The Safe owners need to invoke `disableTimelock()` to disable the activated timelock after the revocation.

File(s) affected: `HypernativeGuard.sol`

Description: The `HypernativeGuard` contract uses a timelock mechanism to prevent an attacker from immediately removing the guard. However, this timelock is only set up to work once. After the timelock has passed and the Safe has revoked the use of the guard, the timelock is not reset. This means that if the Safe were to change the guard in the future, the timelock threshold would have already been passed. Manual intervention can still be used to reset the timelock.

Recommendation: Consider making the following changes:

1. In the `revokingHash` branch of `checkTransaction()`, require that `timelockBlock` is greater than zero and less than `block.timestamp` and set `timelockBlock` back to zero if all conditions are met.
2. In `disableTimelock()`, set `timelockBlock` back to zero.

HNG-5 Missing Ability to Revoke Keeper Role • Low ⓘ Fixed

Update

The client provided the fix in `f2ac43c9afd274f381cb906a4c3083d5160d5407` by adding the ability to revoke keeper roles from the Safe Guard.

File(s) affected: `HypernativeGuard.sol`

Description: The guard includes functionality to grant a keeper role through `grantKeeperRole()`, but lacks a corresponding function to revoke this role. Once granted, keeper permissions cannot be removed without a revoke capability, potentially leading to operational risks in the event of a compromised keeper.

Recommendation: Implement a function to enable revocation of the keeper role.

HNG-6 No Check for Duplicate Policies

• Informational ⓘ

Fixed

✓ Update

The client addressed in `08016d88d824ace1e129d36f7a7b463e32c04679` and `052e01cb170cec824872936f34c81d588d880878` following the recommendation.

File(s) affected: `HypernativeGuard.sol`

Description: The `addPolicyExtension()` function should check to ensure that the address being added is unique. Having duplicate policy extensions will increase gas costs and prevent `removePolicyExtension()` from working correctly. A separate call would be necessary for each duplicate.

Recommendation: Prevent duplicate extension addresses from being added. Alternatively, consider using [OpenZeppelin's EnumerableSet](#) data structure.

HNG-7 Policies Are Not Provided with the Transaction Signatures

• Undetermined ⓘ

Fixed

✓ Update

The client provided the fix in `df70d5db87a01e240e023f6b97951d70f57aa9d1` following the recommendation.

File(s) affected: `HypernativeGuard.sol`, `IGuardPolicyExtension.sol`

Description: As defined in the `IGuardPolicyExtension.sol` interface, a policy extension requires the following parameters:

```
* @dev This function should revert if the transaction violates policy rules
* @param to Destination address of the transaction
* @param value Ether value of the transaction
* @param data Transaction data payload
* @param operation Operation type (Call or DelegateCall)
* @param safeTxGas Gas that should be used for the safe transaction
* @param baseGas Gas costs for data used to trigger the safe transaction
* @param gasPrice Maximum gas price that should be used for this transaction
* @param gasToken Token address (or 0 if ETH) that is used for the payment
* @param refundReceiver Address of receiver of gas payment (or 0 if tx.origin)
* @param signatures Transaction signatures
* @param executor Address of the executor of the transaction
```

However, `HypernativeGuard.checkTransaction()` ignores the given inputs `signatures` and `executor`, passing an empty bytes array and zero address to all policy extensions.

Recommendation: Adhere to the policy extension interface by passing the `signatures` and `executor` parameters from `checkTransaction()` to each policy extension being executed.

Auditor Suggestions

S1 Gas Optimization Consideration

Fixed



Update

The client added custom errors in `1398a36f7ecadb32845b28df60b4780511101202`.

File(s) affected: `HypernativeGuard.sol`

Description:

- Cache Array Lengths in Loops: Store the array length in a local variable before the loop to reduce repeated storage reads.
- Use Custom Errors Instead of String Reverts: Custom errors introduced in Solidity 0.8.27 (without IR feature) can reduce bytecode size compared to revert messages.

Recommendation: Implement the recommended optimizations above to reduce gas consumption in external function calls and loop operations.

S2 Use `supportsInterface()` for Policy Extensions

Fixed



Update

The client provided the fix in `6825cdfa784ace3f8708cf11bea23dbe0997a537` following the recommendation.

File(s) affected: `IGuardPolicyExtension.sol`, `HypernativeGuard.sol`

Description: Any address can be added as a policy extension using `addPolicyExtension()`. However, even a single address that does not have the `checkPolicy()` function will cause a denial of service. By requiring the interface to also implement the `supportsInterface()` function and validating this in `addPolicyExtension()`, it will mitigate the risk of adding erroneous addresses.

Recommendation: Consider adding `supportsInterface()` to `IGuardPolicyExtension.sol` and calling it in `addPolicyExtension()`.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not post an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/Hypernative-Labs/hypernative-guard`

- `37e...366 ./github/workflows/test.yml`
- `9a9...16a ./gitignore`

- 3ed...a22 ./gitmodules
- 957...a1f ./foundry.toml
- c9a...528 ./notes
- 677...85a ./script/GenerateSafeHash.s.sol
- af2...51b ./script/HypernativeGuard.s.sol
- 8cc...b8d ./script/HypernativeGuardDeployment.s.sol
- 989...663 ./src/HypernativeGuard.sol
- e55...2cb ./src/IGuardPolicyExtension.sol
- 259...dcc ./test/HypernativeGuard.t.sol
- 4b4...5b3 ./test/Mocks/MockPolicies/AddressZeroNotAllowedPolicy.sol
- b71...288 ./test/Mocks/MockPolicies/DelegateNotAllowedPolicy.sol
- 29a...f97 ./test/Mocks/MockPoolManager.sol
- 669...0c5 ./test/SigUtils.sol

Repo: https://github.com/Hypernative-Labs/hypernative-guard

- 37e...366 ./github/workflows/test.yml
- 9a9...16a ./gitignore
- 3ed...a22 ./gitmodules
- 957...a1f ./foundry.toml
- c9a...528 ./notes
- 677...85a ./script/GenerateSafeHash.s.sol
- af2...51b ./script/HypernativeGuard.s.sol
- 8cc...b8d ./script/HypernativeGuardDeployment.s.sol
- 8f9...462 ./src/HypernativeGuard.sol
- e55...2cb ./src/IGuardPolicyExtension.sol
- 259...dcc ./test/HypernativeGuard.t.sol
- 4b4...5b3 ./test/Mocks/MockPolicies/AddressZeroNotAllowedPolicy.sol
- b71...288 ./test/Mocks/MockPolicies/DelegateNotAllowedPolicy.sol
- 29a...f97 ./test/Mocks/MockPoolManager.sol
- 669...0c5 ./test/SigUtils.sol

Files

- 4f1...4e4 ./src/HypernativeGuard.sol
- a2a...193 ./src/IGuardPolicyExtension.sol

Tests

- 150...658 ./test/SigUtils.sol
- 310...72e ./test/HypernativeGuard.t.sol

Toolset

The notes below outline the setup and steps performed in the process of this audit.

Setup

Tool Setup:

- [Slither](#)  v0.11.0

Steps taken to run the tools:

1. Install the Slither tool: `pip3 install slither-analyzer`
2. Run Slither from the project directory: `slither .`

Automated Analysis

Slither

We have executed slither and filtered the issues that were reported and incorporated the valid ones in the report. Please note that only issues related to the scope of the audit are reported.

Test Suite Results

```
Ran 16 tests for test/HypernativeGuard.t.sol:HypernativeGuardTest
[PASS] testConfigureHypernativeGuard() (gas: 107884)
[PASS] testRevokeHypernativeGuardRevertsOnTimelockInit() (gas: 185961)
[PASS] testRevokeHypernativeGuardRevertsOnTimelockStillActive() (gas: 282834)
[PASS] test_ApproveAndRevokeNonceFree() (gas: 241471)
[PASS] test_ConfigureHypernativeGuardAndAbortUnapprovedTx() (gas: 159069)
[PASS] test_ConfigureHypernativeGuardAndExecuteWithdrawTx() (gas: 197601)
[PASS] test_FunctionCallHash() (gas: 341572)
[PASS] test_GrantAndRevokeKeeperRole() (gas: 404392)
[PASS] test_NonceFreeTx() (gas: 326463)
[PASS] test_PolicyExtension() (gas: 581833)
[PASS] test_RevokeApprovedHashExpectRevert() (gas: 180990)
[PASS] test_RevokeHypernativeGuardSuccessfully() (gas: 310125)
[PASS] test_SendFunds() (gas: 18372)
[PASS] test_TimelockCancellation() (gas: 276100)
[PASS] test-WithdrawEth() (gas: 90611)
[PASS] test_addHashNotKeeperReverts() (gas: 11281)
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in 1.55s (1.31s CPU time)

Ran 1 test suite in 1.56s (1.55s CPU time): 16 tests passed, 0 failed, 0 skipped (16 total tests)
```

Code Coverage

The test suite relies on deployed Safe instances, requiring access to an owner's private key, which complicates the test suite. We modified the test suite setup to deploy a Safe locally to collect test coverage.

Fix Review: During the fix review, the client refactored some tests and added additional assertions to check for the expected error. We recommend that the client update the test suite to reference a local Safe deployment to simplify the test setup.

File	% Lines	% Statements	% Branches	% Funcs
src/HypernativeGuard.sol	96.25% (77/80)	95.71% (67/70)	77.78% (14/18)	95.45% (21/22)
Total	96.25% (77/80)	95.71% (67/70)	77.78% (14/18)	95.45% (21/22)

Changelog

- 2025-03-28 - Initial report
- 2025-04-15 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

